



全球软件质量&效能大会 · 上海站

GLOBAL SOFTWARE QUALITY & EFFICIENCY CONFERENCE · SHANGHAI



Recursive Self-improvement Agent Harness: 自我迭代的 Harness Engineering

罗正昕 | 百度 Comate Agent 策略负责人



罗正昕

百度 Comate Agent 策略负责人

目前在百度 Comate 团队负责 AI Coding Agent 相关算法与系统的研发，专注于大模型 Agent 的复杂任务执行能力和工程化落地。

主要负责：

- Comate Agent 新一代 Harness 的设计与构建
- Coding Agent 自动化评测体系的建立
- Agent Harness 自我迭代系统的构建

目录

CONTENTS

01

从 Vibe Coding 到 Agentic 研发的
范式转变

02

Agentic 研发的核心关键——
Harness 层

03

Harness 层研发面临的核心困难

04

自我迭代 Harness 方案

05

总结

PART ONE Agentic 研发范式

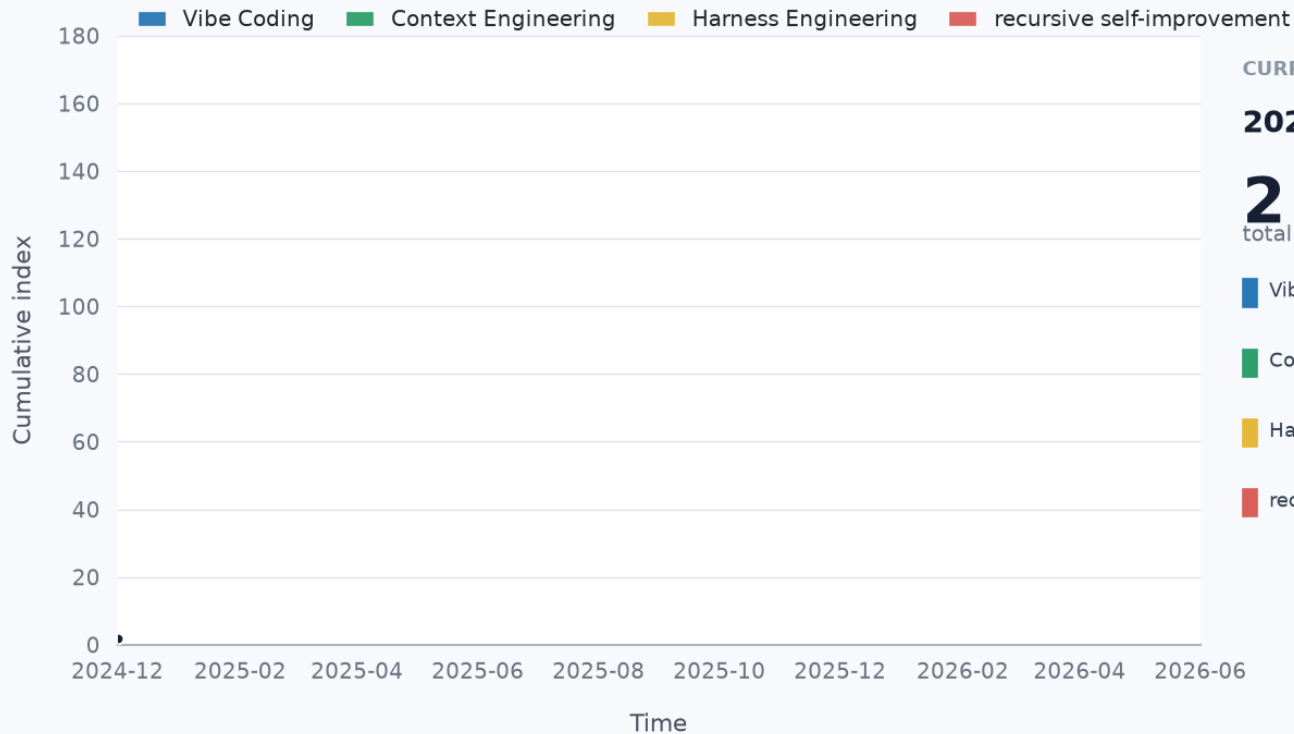
从 Vibe Coding 到 Agentic 研发的范式转变

01



QE Vibe Coding 的兴起

Google Trends · Worldwide



CURRENT SNAPSHOT

2024-12-01

2
total index

Vibe Coding	0
Context Engineering	2
Harness Engineering	0
recursive self-improvement	0

QE Agentic 研发范式

Vibe Coding

是从“写代码”转向“表达意图”

Agentic Engineering

则是从“让 AI 写代码”转向“让 AI 自主完成工程任务”

The evidence we've laid out here suggests that we're likely heading into this scenario. But speeding up one part of a process often just shifts the bottleneck elsewhere: overall pace is capped by the parts that haven't sped up. In computing, this is known as Amdahl's law, and the same logic can apply to organizations. Anthropic has already encountered one signature of Amdahl's law: as we've begun to push more code around the organization, human code review has become a new bottleneck.

We've also encountered this friction outside engineering. There has been an explosion of new ideas, initiatives, tools, and simulations, as a result of Anthropic employees working with highly capable models—far more than we have the capacity to pursue. The rate at which organizations can spot and

Amdahl's 定律：当 AI 编写了更多的代码时，人工的测试、验证、审核成为了新的瓶颈。

所以 AI Agent 需要在更长程的条件下完成端到端的自动化验证、测试等工作

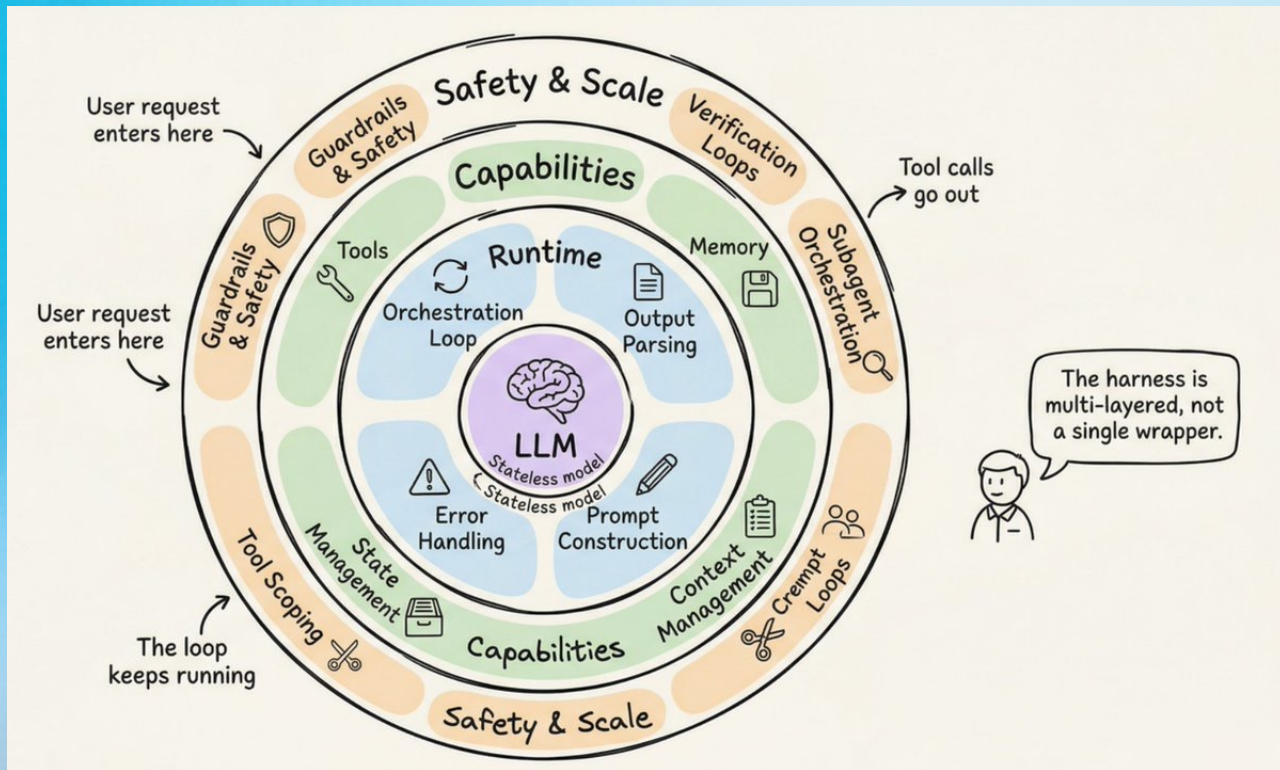
PART TWO Harness 层

Agentic 研发的核心关键——Harness 层

02



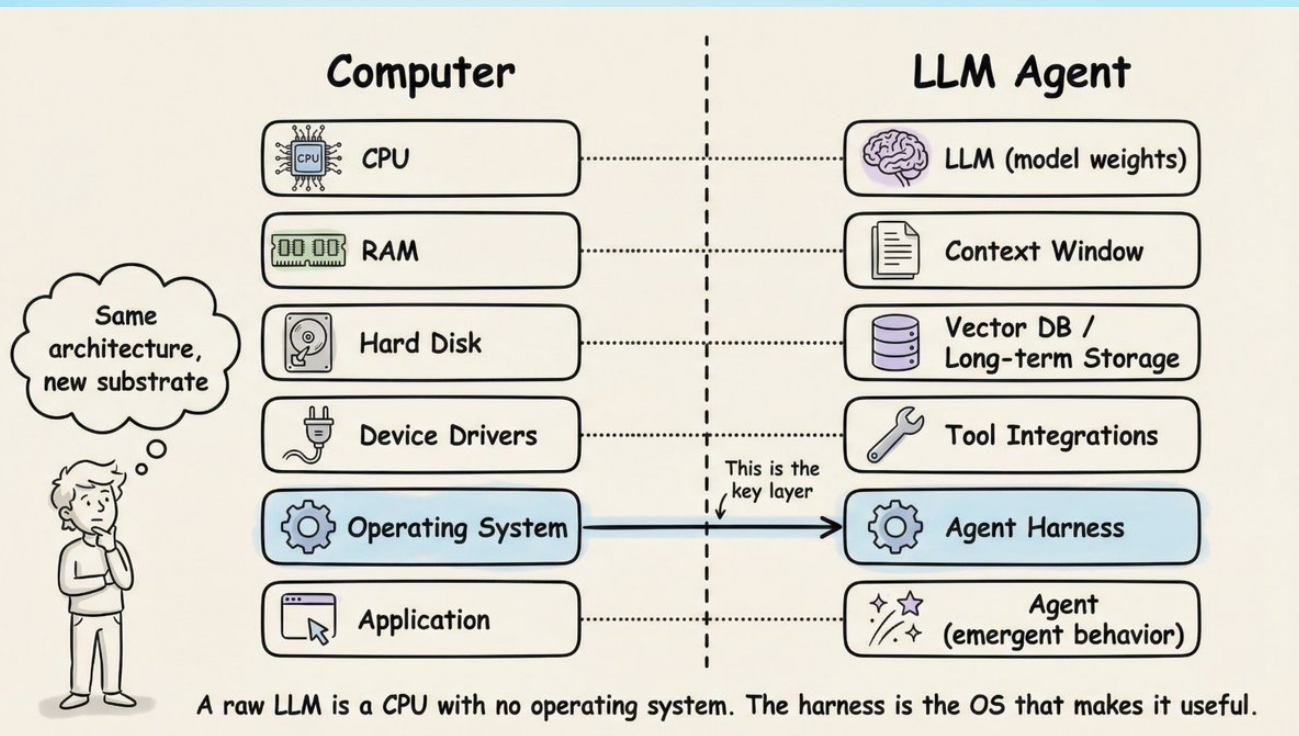
QE 什么是 Harness 层



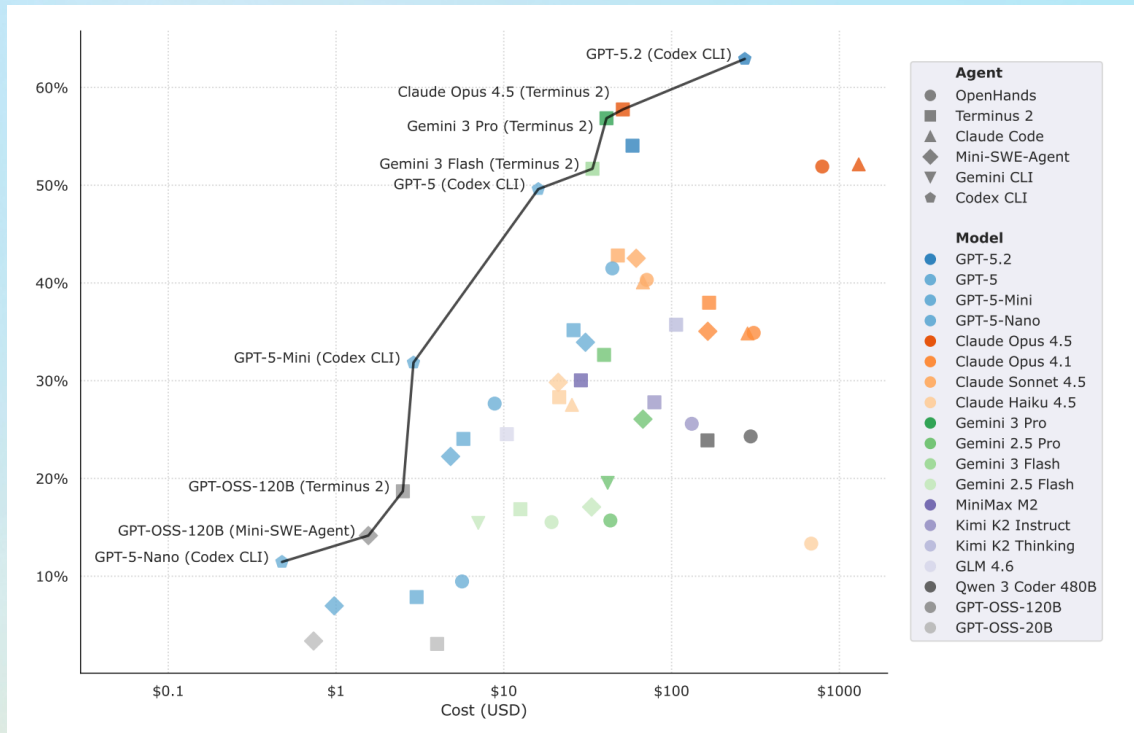
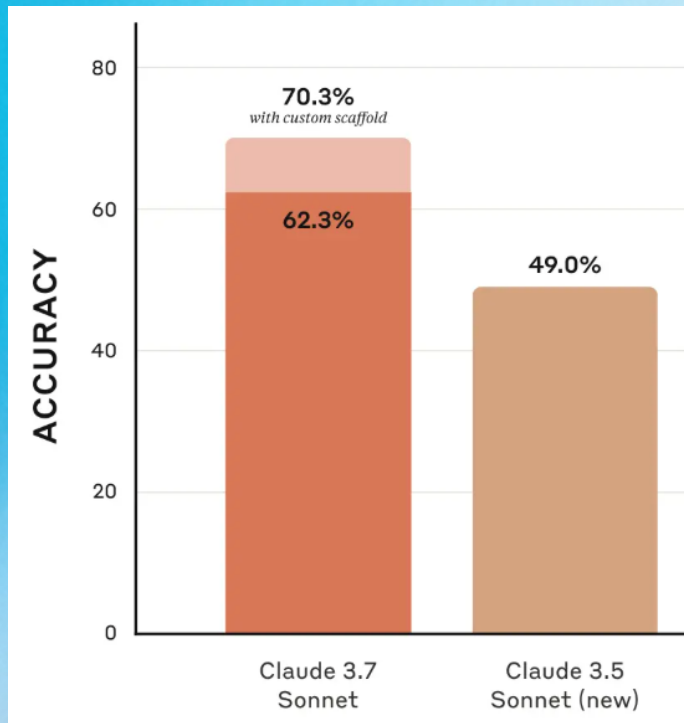
Harness 层是包围在 LLM 外面、让 Agent 能够可靠完成工作的那一套“工程基础设施”。

QE Harness 层的功能

Harness 层就相当于 LLM Agent 的操作系统。



QE Harness 层为什么重要



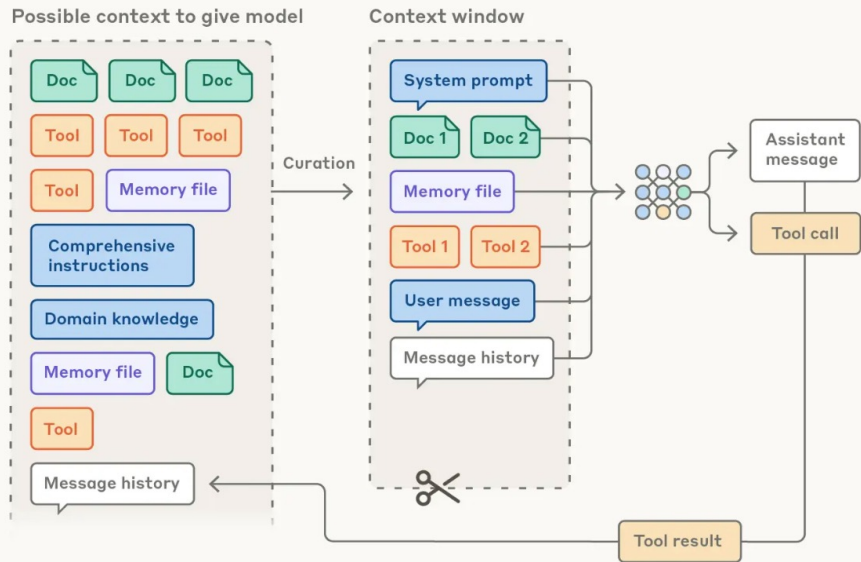
即便使用完全相同的模型，不同的 Agent Harness 的最终得分也有很大差异

1. <https://www.anthropic.com/news/claude-3-7-sonnet>

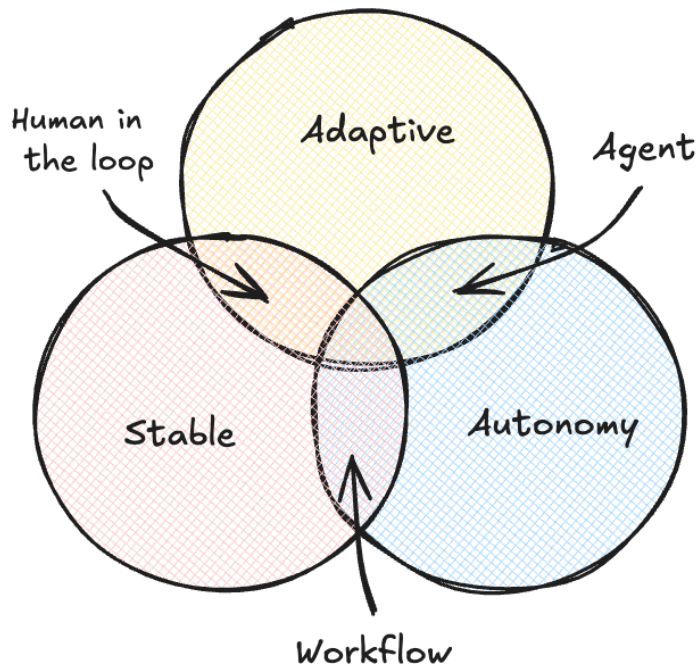
2. Merrill, Mike, et al. "Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces." *International Conference on Learning Representations*. Vol. 2026. 2026.

QE 什么是好的 Harness 层

Context engineering for agents



好的 Harness 层就是在尽可能平衡其中需求



Harness 层的本质是管理好注入模型的上下文

QE Harness 层的发展方向

时间	任务复杂度	典型任务	产品形态
2024 前	20 min	给定修改意图，然后看一段代码后完成修改	Completion / Chat
2025	1 hour	实现一个 Feature 的完整内容	Sync Local Agent
2026	6 hour	对齐需求后，实现一个 Feature 的完整内容，并进行完整测试验证，有问题修复，直至达到预定目标	Ralph Loop Agent Async Remote Agent
预测	30 hour	多 repo 跨库开发联调测试	产品级 agentic workspace 研发环境

最佳产品形态以及 Harness 层形态和模型能够处理任务的复杂程度相关

Scaling 存在上限，因为前期需求讨论以及最终结果检验仍然需要人重度参与 → 新产品形态

讨论研发需求的工具（主动式）

审核 Agent 实现过程及测试结果的工具（责任反转）

PART THREE Harness 层研发 核心困难

Harness 层研发面临诸多困难

03



QE Harness 层研发面临的问题：效果由多重因素决定

正确无严重 bug 的实现是 Agent 能力的保障

得益于模型的泛化能力，当你的 Harness 层中有一些小的 bug 时，模型仍然能“看似正常”的运行，甚至能在之后猜测出运行的结果。

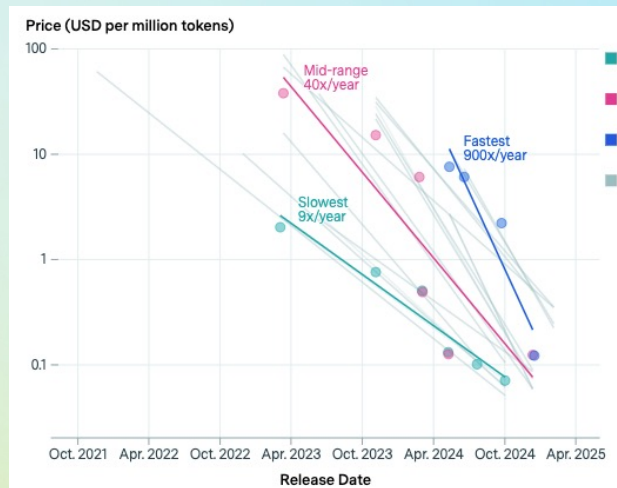
举例：当我们写的工具定义存在问题时，模型调用工具失败后，它能从失败提示中学会怎样去适应这个 bug，从而让整个流程继续运行。但是这些消息都给上下文注入了无关的噪声，从而降低最终效果。

良好的上下文管理能最大程度的发挥模型的潜能

不同上下文管理方案对于 Agent 能力的影响远比想象的大

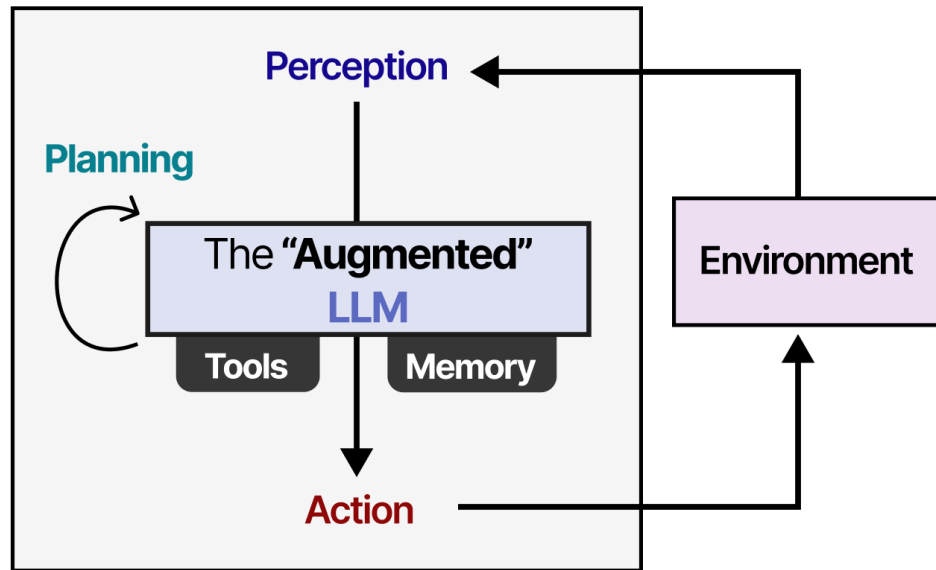
优秀的基础模型是 Agent 能力的来源

不要浪费时间在能力一般的模型上：当前模型能力的发展速度，让我们必须超前考虑模型的最新能力。我们需要根据自己预估的研发及推广速度来考虑选取最终使用的基础模型



QE Harness 层研发面临的问题：难以复现

Agent



在真实使用过程中，Agent 的行为并不只由模型和 Prompt 决定，还受到大量本地环境因素影响：

- 用户使用的操作系统、IDE、Shell、权限配置
- 当前工作目录和项目目录结构
- 本地文件内容、未提交代码、配置文件、环境变量
- 可用的工具、依赖包、版本
- MCP 服务
- 用户在过程中进行的人工干预
- 机器资源受限、抢占、冲突等
-

遇到问题时研发人员极难复现

QE Harness 层研发面临的问题：归因困难

缺少可解释的效果衡量：

System Prompt、Tool Description、Tool Result大量文本内容，且没有具体效果衡量，什么文本能够激活什么行为模式无固定结果。

模型的推理具有随机性：

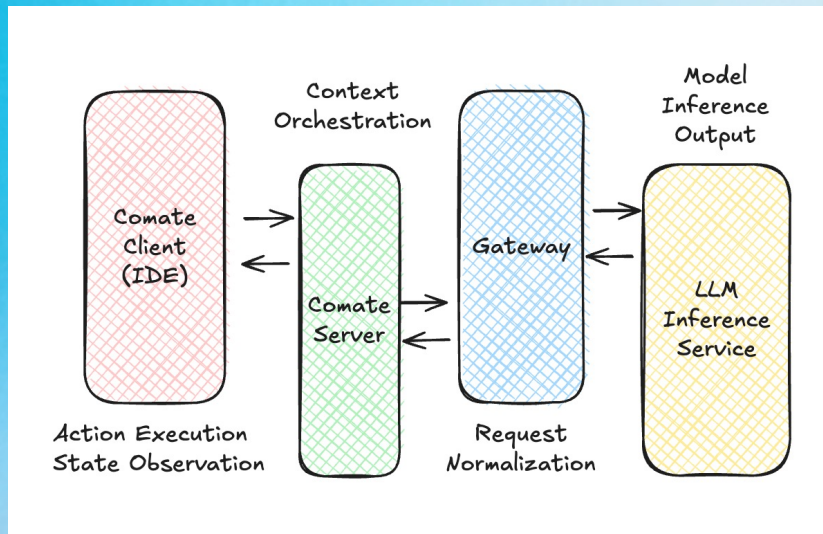
轨迹上出现单次可能的失效模式不一定会重复发生

Badcase 作用机理复杂：

System Prompt 中规定的行为模式、工具执行结果、上下文压缩、subagent 调用编排都会彼此影响

QE Harness 层研发面临的问题

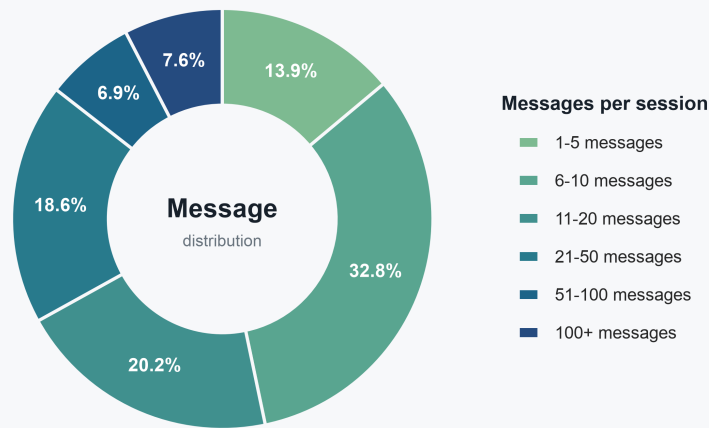
长链路 + 长轨迹导致追溯困难且定位成本高昂



每次请求都要经过极长的链路得到结果

链路上的任何一环都有可能出现问题

Session Message Distribution



几十轮消息叠加单轮多个工具并行调用形成的长轨迹

在其中难以定位的具体问题

QE Harness 层研发面临的问题：难以稳定回归

Agent 的优势

Agent 能够通过尝试、观察、反思、再次尝试的循环，通过多步迭代求解复杂问题。

Agent 特有的适应性使得同一个 Agent 能够适应非常广泛的场景、非常极端的边界条件。

导致

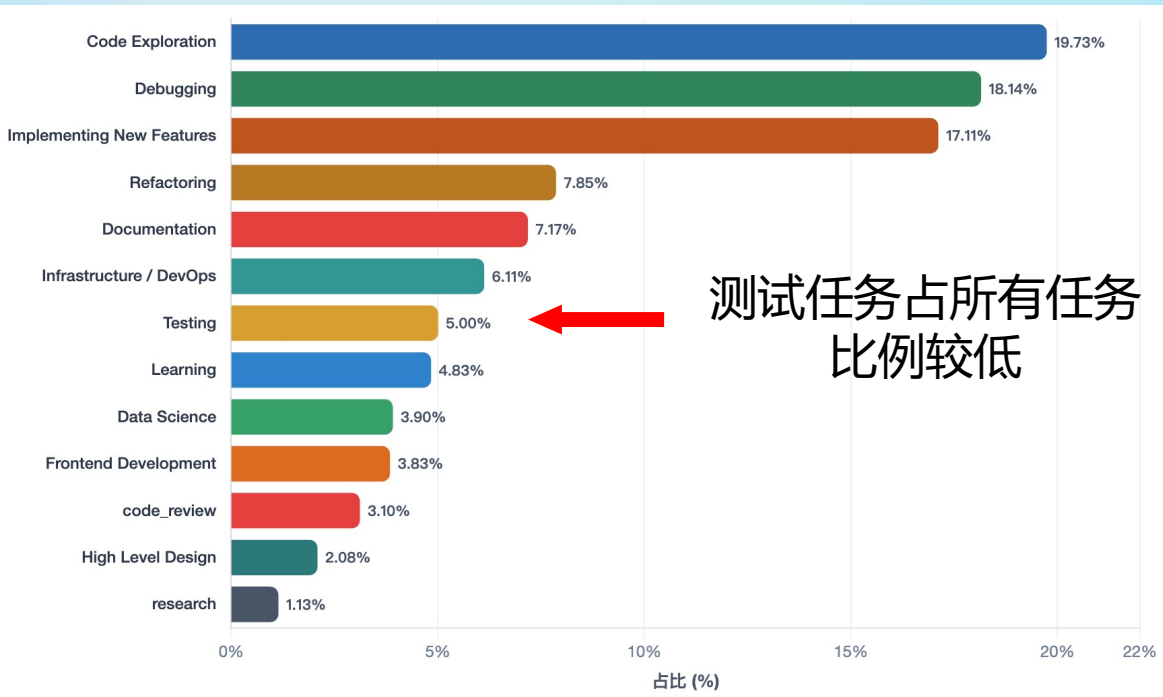
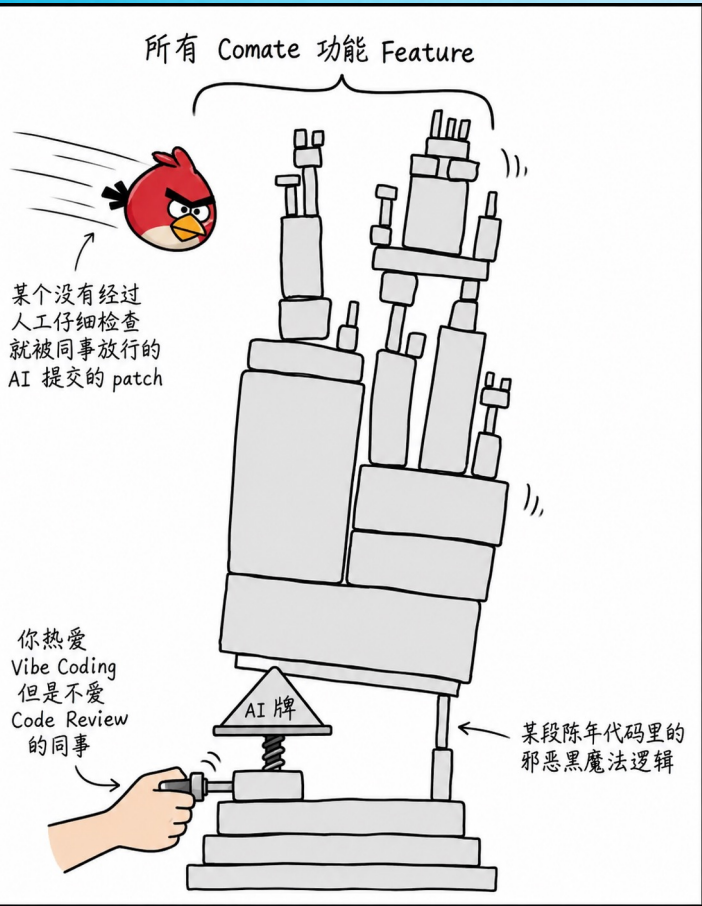
Agent 执行链路长，单条轨迹的测试对于人力成本、时间成本、模型推理成本都是巨大的开销。

想要保证对 Agent 的修改没有对于其他场景的性能降低需要覆盖非常广泛且多样的场景。

改完以后难以进行稳定的回归测试

QE Harness 层研发面临的问题：快速迭代

在 Agent 研发的时代，有些问题将变得更加棘手.....





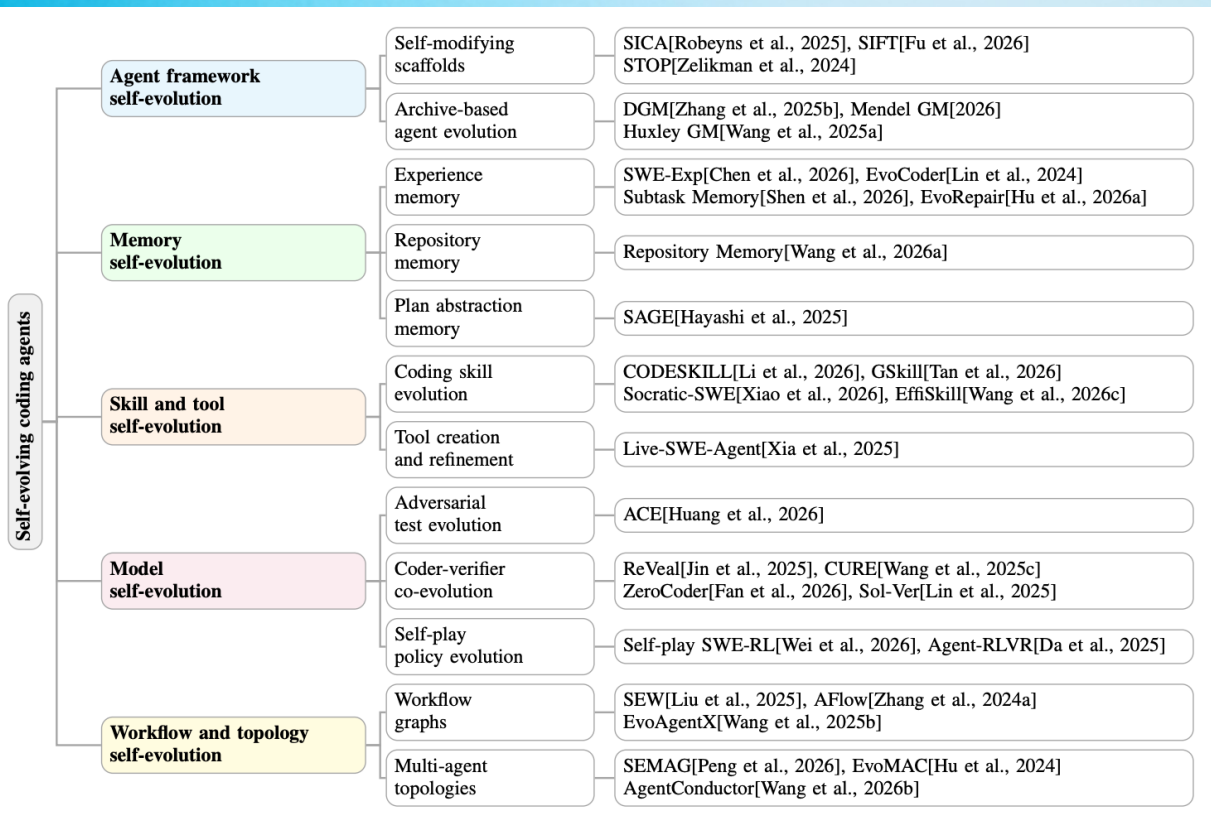
PART FOUR 自我迭代

让 Harness 层的迭代能够自动循环起来

04



QE Recursive Self-improvement



学界已有一些研究成果:

1. Agent 框架的优化

2. 基于记忆的优化

3. 优化 skill 或是工具集

4. 模型层的自我迭代

5. Agent | Workflow 各模块、subagent之间的结构关系

QE Recursive Self-improvement

问题分析

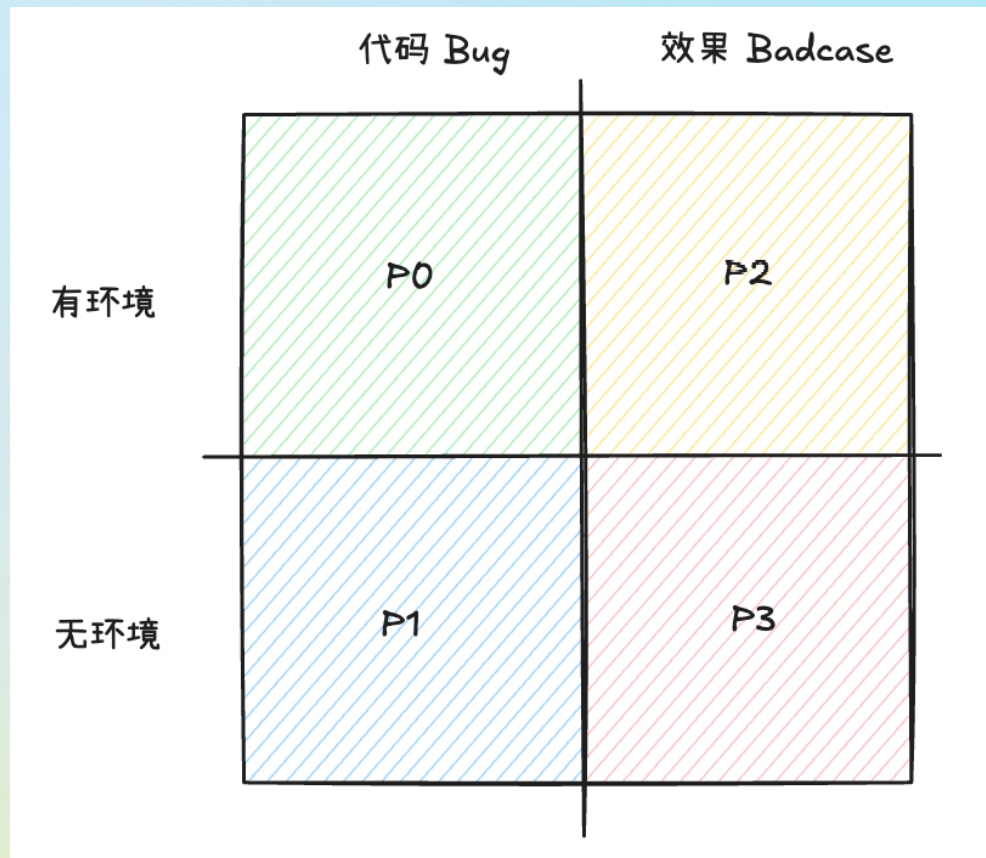
有无环境意味着是否可以较为方便的复现

问题如果是代码 Bug，意味着复现时针对固定的输入可以产生确定性的状态

问题如果是效果 Badcase 意味着复现时针对固定的输入可能会产生随机的效果



迭代指标是否明确



QE Recursive Self-improvement

有环境的代码 Bug

如评测环境中发现的代码 Bug，最容易复现及验证

无环境的代码 Bug

如用户使用时发现的代码 Bug，可以通过日志记录等方式排查。较容易复现，但是不太容易验证

有环境的效果 Badcase

如评测时发现的未解决、分数低的 case，不太容易复现，但是相对容易验证

无环境的效果 Badcase

如用户使用时效果不及预期，不太容易复现和验证

多端日志串联

Agent 应用的使用链路非常长，排查时需要联合多端日志统一排查，所以需要打通多端日志，统一串联起来。且能够统一进行访问读取。

日志归一化

不同的日志中所记录的名词不一致，比如各端对于 Session ID 的定义存在不同，但是多端日志查询的时候需要 Agent 比对查看，所以我们需要进行一定的后处理。

用户反馈机制

结合办公 IM 软件，让用户反馈能够更便捷的在处理用户问题的同时自动积累对应测试用例，转化为后续迭代的测试集。

评测系统

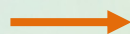
对于效果类的需求，建立一个高效稳定的评测 Benchmark 评测系统是非常重要的，效果类的改进都依赖于评测系统的指标反馈。

QE 基建准备

因为 Comate 是前后端分离的架构，所以除了接口是两方代码库所统一对齐的，不同代码库里具体实现的时候可能有很多的内部概念。

这些情景需要让 Agent 重新梳理映射关系，包括：

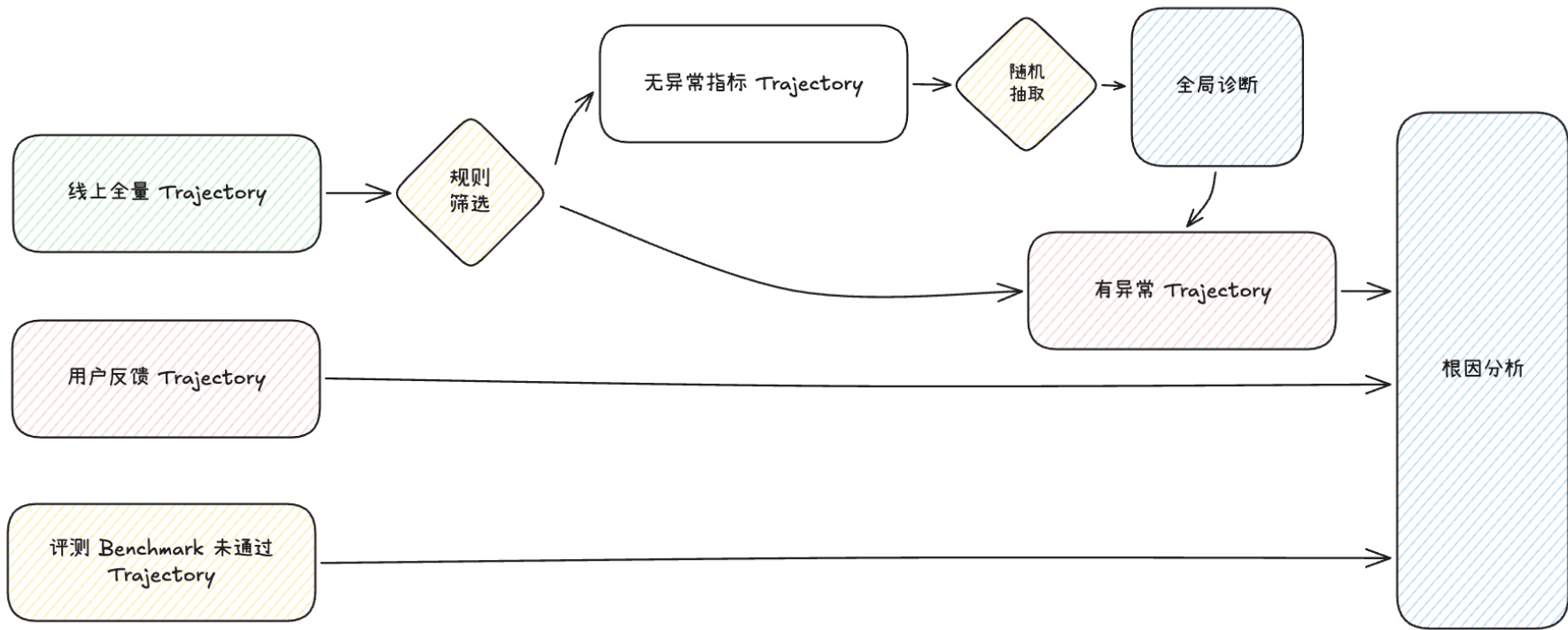
- 同概念在不同代码库拥有不同名字
- 不同概念在不同代码库拥有相同名字



在每个代码库中 docs 目录下增加跨代码库联合开发易混淆术语文档



QE 迭代原始驱动力——数据来源



QE 迭代原始驱动力——数据来源

线上数据的筛选规则漏斗：

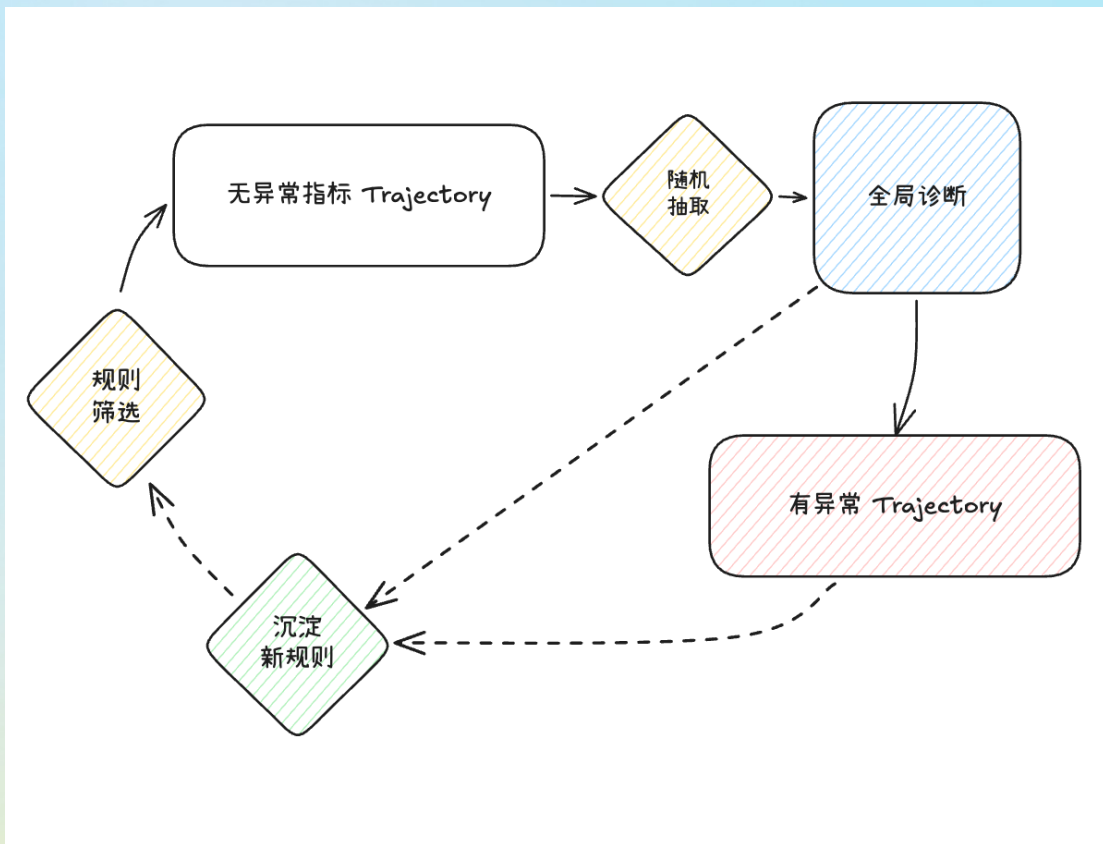
- 某条数据偏离于正常阈值的
- 某步数据偏离于正常阈值的

筛选漏斗规则决定了后续发现能力

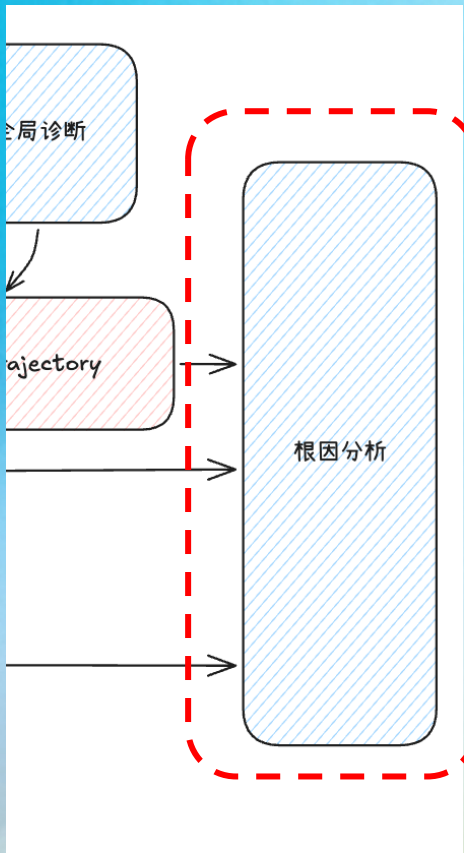
初期规则
不完善

适用规则
漂变

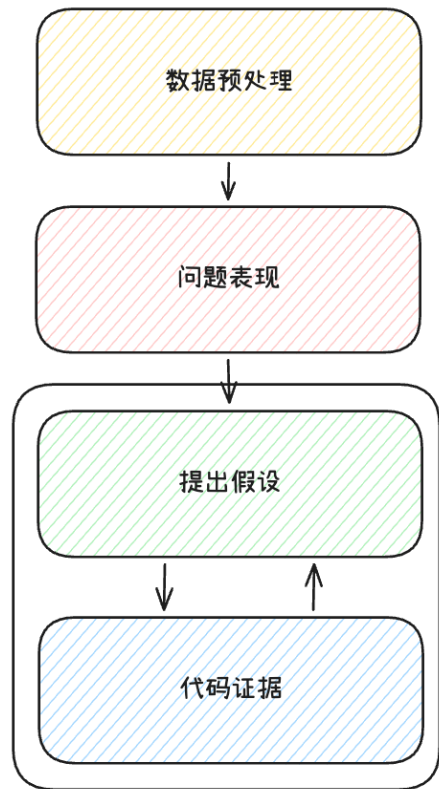
迭代更新漏斗规则

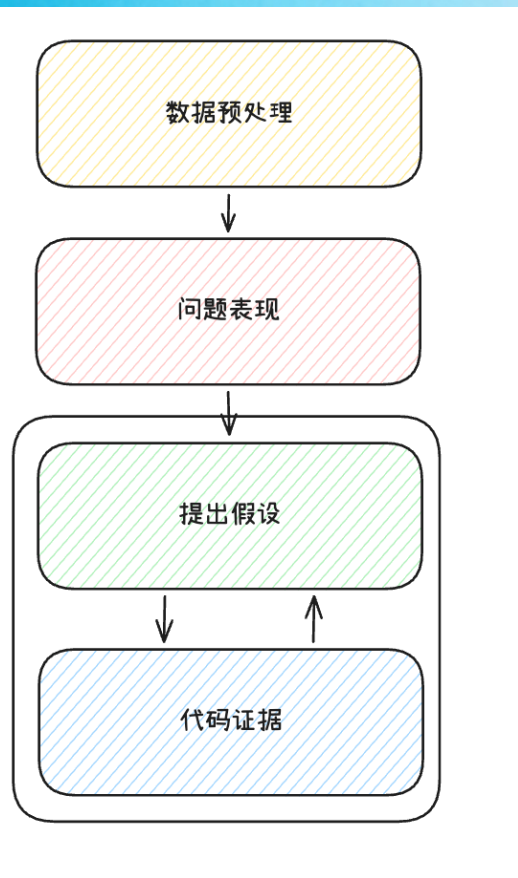


QE 根因分析模块



QE 根因分析模块





遇到的问题

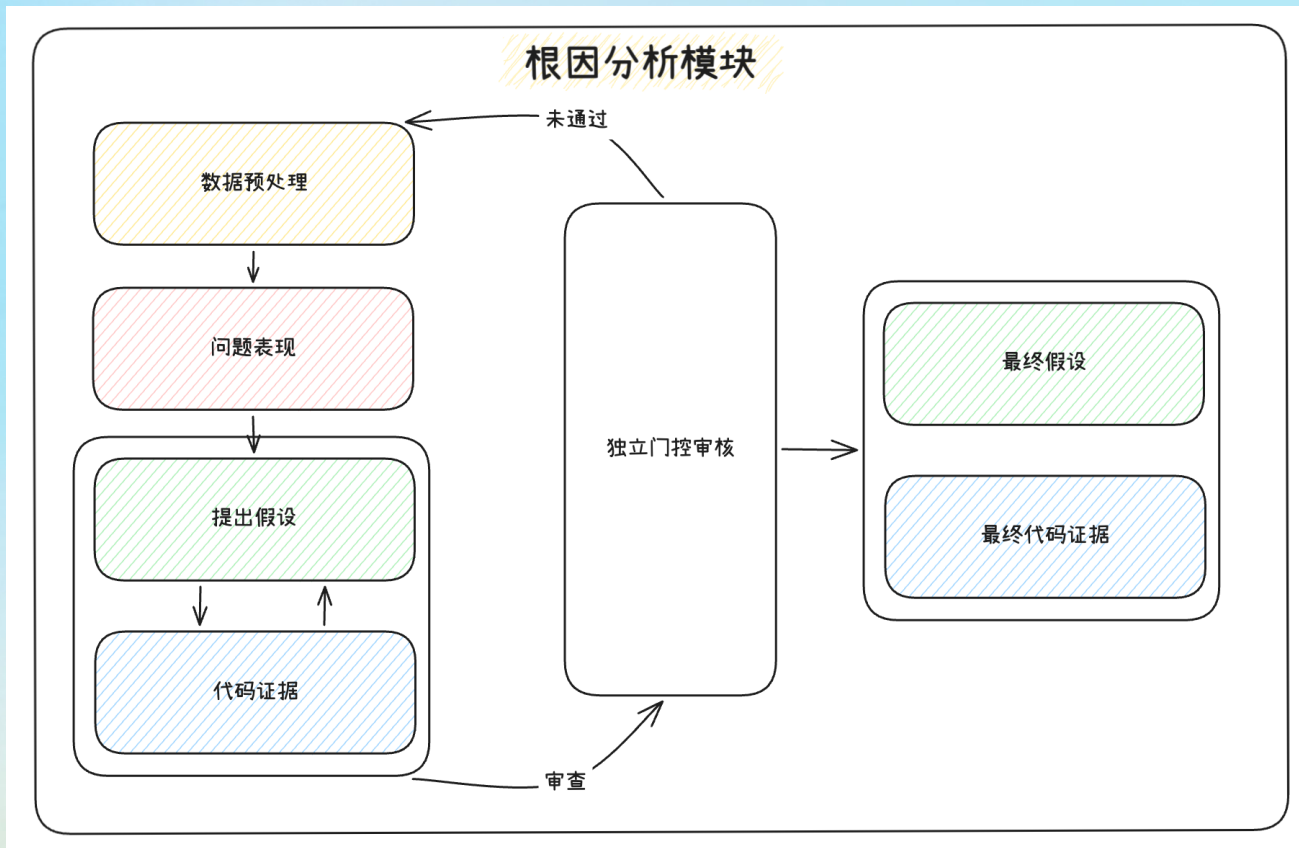
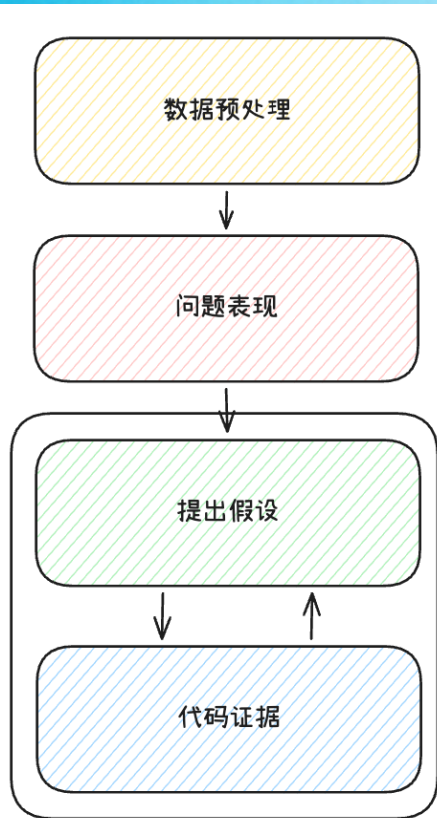
准确性低

分析出来的结果
和人工核验结果
差别过大

报告产出不稳定

同样的问题描述和
Agent 执行轨迹所
得到的根因分析报
告每次都不一样

QE 根因分析模块



增加门控机制，由独立 Agent 完成审核

QE 修复模块

单独上下文窗口

根因分析时已经使用了部分上下文窗口，修复往往也涉及多轮迭代，所需上下文窗口较大

典型失败模式纠正

Agent 进行修复时存在一些固定失败模式，此时我们需要补充对应规则约束、指导意见

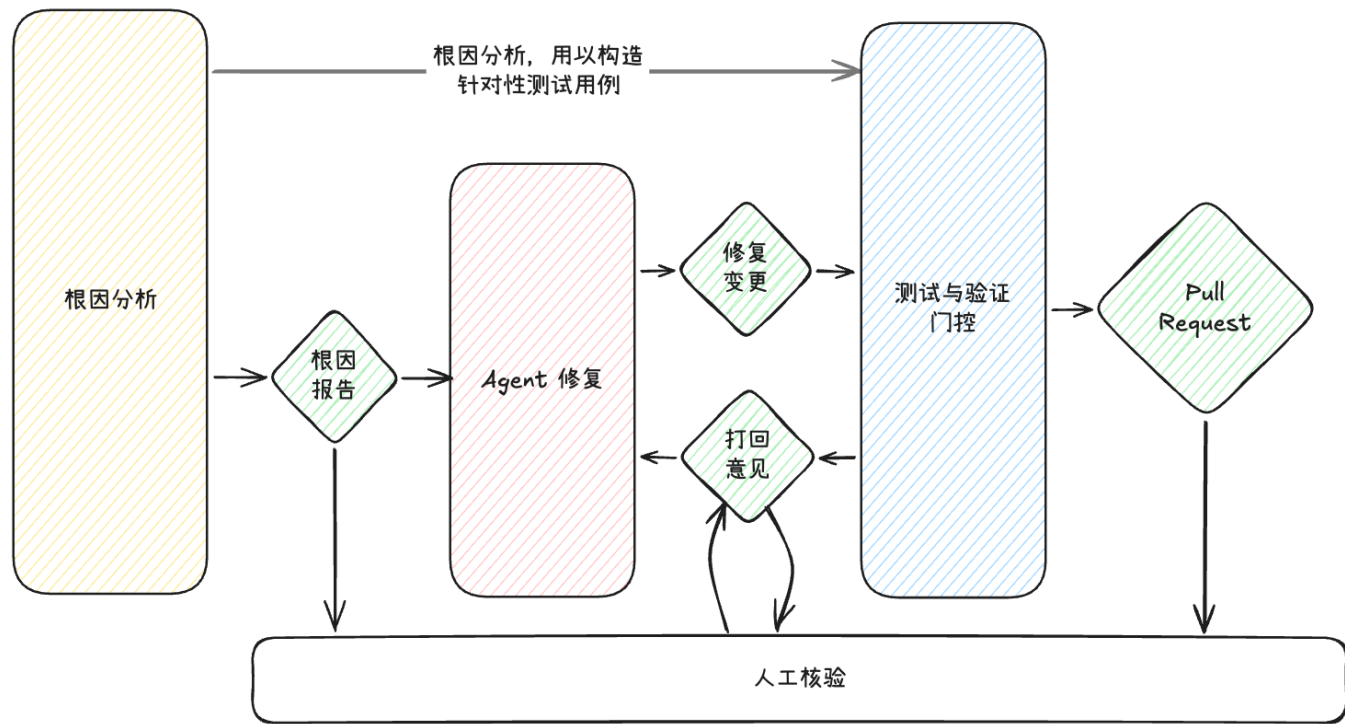
代码组织结构偏好注入

代码最终还是需要人来负责最终兜底维护，为了让人能够更好维护，需要有更多的组织偏好意见

上下游阶段解耦

可以分步骤迭代，彼此不耦合。且修复前、修复后可以增加人工审核

QE 测试模块-代码 Bug



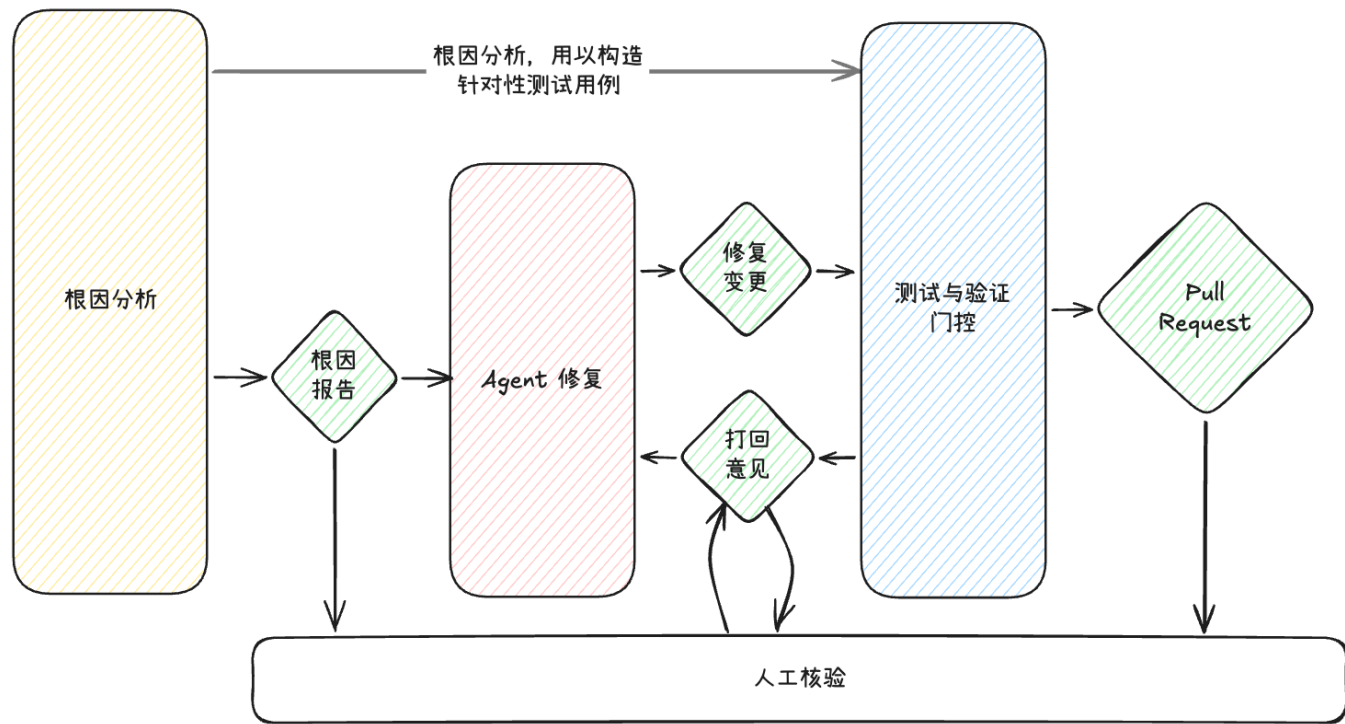
单元测试+静态检查

核心主流程端到端
测试 Case

根因报告针对性
构造测试 Case

根据代码变更针对
性构造测试 Case

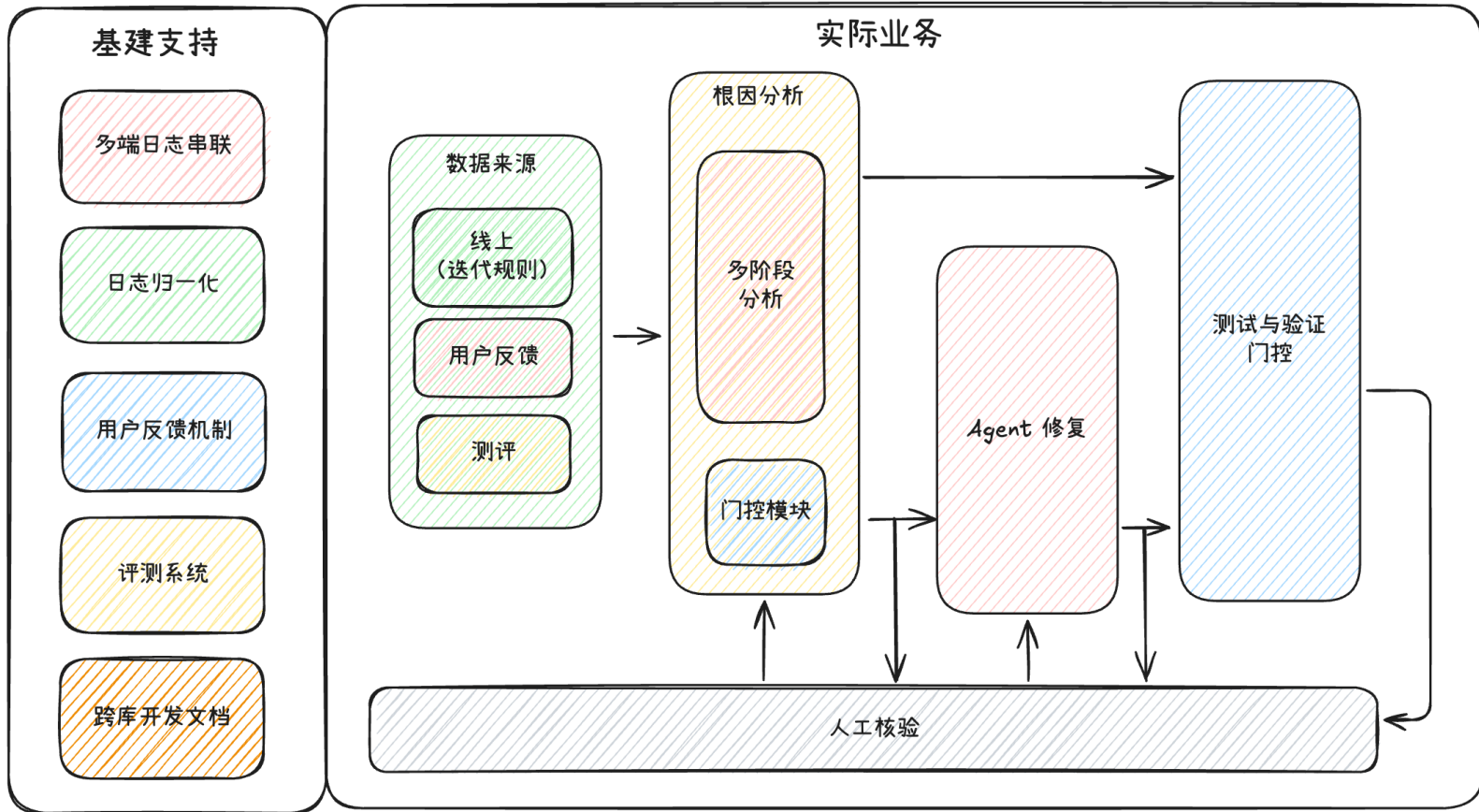
QE 测试模块-效果类 Badcase



单元测试+静态检查

核心主流程端到端
测试 Case

Benchmark 再次验证

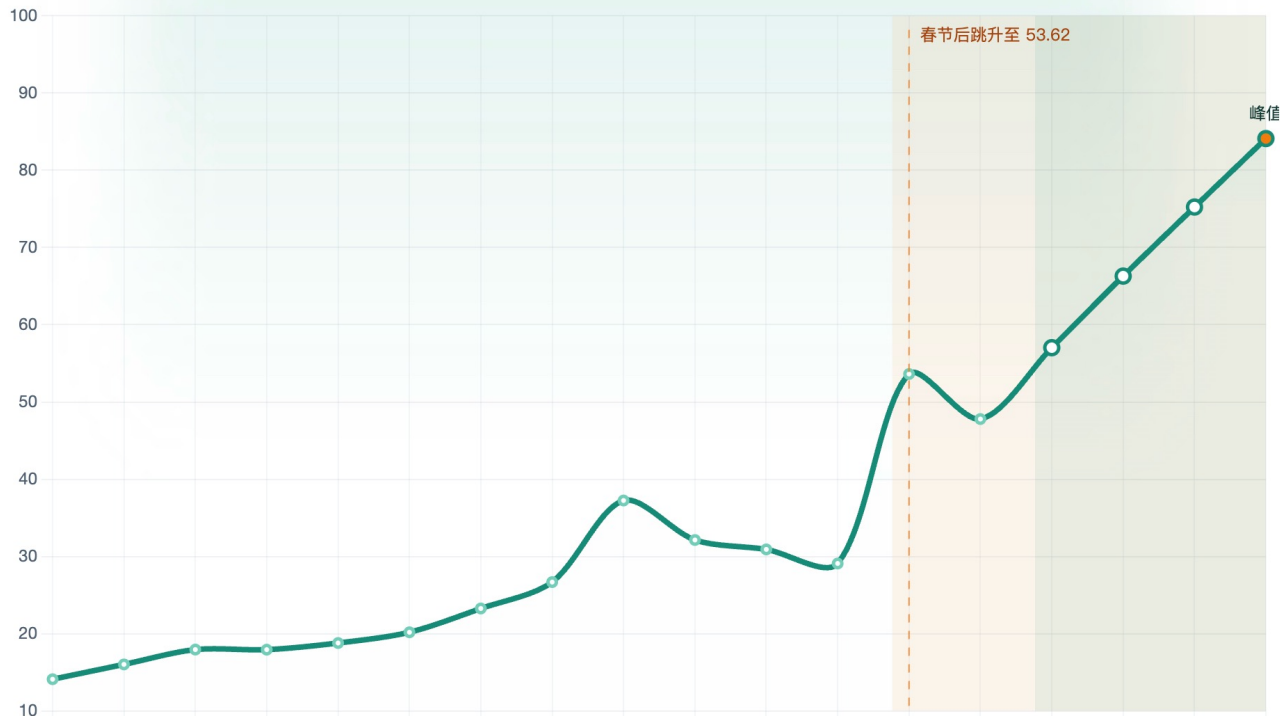


QE Coding Agent 在百度落地的效果

增长趋势图

单图突出快速增长，标记 2 月末跳升拐点与 3 月持续创新高区间。

18 个周度观测点



人均 Query 次数
增长5倍

相对于人均Tokens统计，
我们更专注与人均
Query次数统计，有效、
合理的Query更能代表
用户对Coding Agent的
使用情况。



PART FIVE

总结

本次分享总结

05



QE Take away

Vibe Coding 转向 Agentic 研发

Agentic 研发的核心：Harness 层

Harness 层研发的难点

- 效果由多重因素决定
- 难以复现
- 长链路+长轨迹导致追溯困难且定位成本高
- 归因困难
- 难以稳定回归
- 快速迭代时代的新问题

Harness 层自我迭代

问题分析：通过迭代指标是否明确确定优先级

基建准备：多端日志串联、日志归一化、用户反馈机制、评测系统、跨代码库开发文档

数据来源：线上、用户反馈、评测 Benchmark

根因分析模块：准确性低和不稳定→门控机制

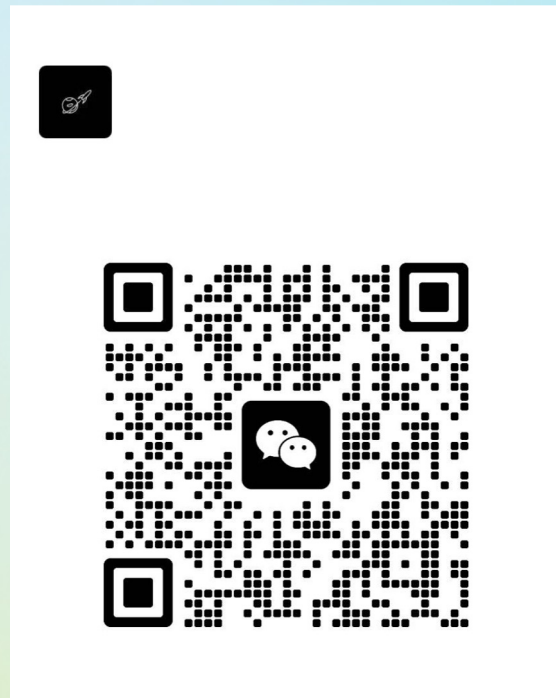
修复模块：单独的 Agent，为其构建修复规则约束与指导意见

测试模块：单元测试及静态检查做基础，针对性测试/Benchmark 评测增强

PART SIX

Q&A

欢迎大家加我微信，会后继续交流



QECon

全球软件质量&效能大会

感谢聆听



关注公众号



查看更多会议详情